

Chapitre 1 : CODAGE DE L'INFORMATION

Objectifs spécifiques

- Comprendre la notion d'information et le codage de l'information
- Connaître les types et les classes d'information dans un ordinateur
- Coder (Représenter) dans un ordinateur les entiers naturels, relatifs, les nombres réels, les caractères et les chaînes de caractères
- Distinguer la notion d'écriture d'un nombre décimal en binaire de sa représentation dans un ordinateur
- Effectuer des opérations arithmétiques en binaire

Eléments de contenu

1. La notion d'information
2. Les types d'information dans un ordinateur
3. La classification des informations traitées par un l'ordinateur
4. Le système binaire
5. Le codage (la représentation) de l'information dans un ordinateur
6. Le codage des entiers non signés (entiers naturels (entiers positifs ou nuls))
7. Le codage des entiers signés (entiers relatifs (entiers positifs et entiers négatifs))
8. Le codage des nombres réels
9. Le codage des caractères et les chaînes de caractères ;
10. Les autres codes binaires
11. Les opérations arithmétiques en binaire

Volume Horaire :

Cours : 4 heures

TD : 3 heures

1.1 NOTION D'INFORMATION

1.1.1 De la perception de l'humain

L'information est la mesure de la densité d'un renseignement contenue dans un message. Celle-ci se présente sous plusieurs formes, à savoir :

- **L'information écrite ou littérale** comprenant des signes utilisés dans l'écriture : lettres de l'alphabet, chiffres décimaux, caractères spéciaux ; l'information élémentaire est le caractère.
Exemples : un rapport, une facture, un chèque, ...
- **L'information visuelle ou graphique** est formée par les images et les graphiques. La plus petite information étant le dessin d'une surface pouvant contenir une lettre, un chiffre, ou une forme.
Exemples : le tableau de bord d'une voiture, un graphique, un dessin, ...

- **L'information sonore** est constituée par le son. Notons que le son doit être converti en un signal électrique dont les principales caractéristiques sont la fréquence, l'intensité ou volume, et la durée.
- **L'information tactile** est basée sur le toucher, par exemple, l'information de type braille utilisée par les aveugles.
- **L'information olfactive** se base sur l'odeur. Exemple d'un atelier de production chimique.

1.1.2 Du point de vue de l'ordinateur

Dans un ordinateur, l'information existe sous deux principales formes :

- Les instructions constituées par les ordres d'un programme informatique ;
- Les données, les objets sur lesquels agissent les instructions. Ces données sont en outre composées de données numérique, alphanumérique (caractères), et de type image, son ou vidéo.

Le schéma ci-dessous donne la classification des informations dans un ordinateur.

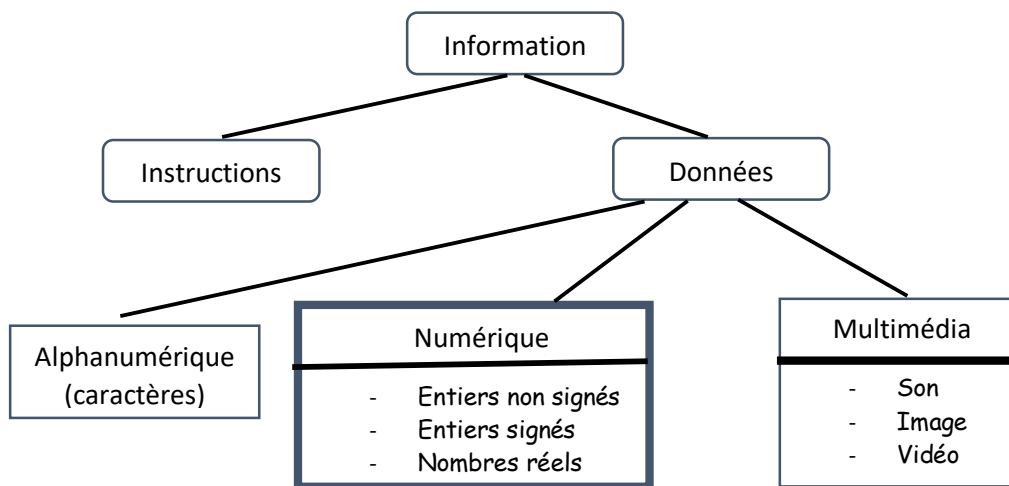


Figure 1.1 : Classification des informations

1.2 LE SYSTEME BINAIRE

1.2.1 Définition du système binaire

Le système binaire fut introduit par Leibnitz au XVII^{ème} et se définit comme suit :

- Chaque position vaut 0 ou 1 avec toute position venant juste à gauche de la position précédente 2 fois celle-ci ;
- La première position à droite est celle de poids nul (unités), ou de poids faible, la précédente celle de poids 1 (2), et ainsi de suite ... ;
- La position la plus à gauche est celle de poids (n-1), ou poids fort où n est le nombre de bits de représentation.

La figure ci-après donne le codage binaire sur 4 bits des chiffres décimaux (0-9)

0	1	2	3	4	5	6	7	8	9
0000	0001	0010	0011	0100	0101	0110	0111	1000	1001

Figure 1.2 : Représentation binaire des chiffres décimaux sur 4 bits

1.2.2 Notions de bit et de mot

Le bit ou Binary Digit □

Le bit ou binary digit est l'information élémentaire (la plus petite unité d'information). Elle vaut soit 0, soit 1 et correspond à un élément électronique ne pouvant prendre que deux états (deux niveaux de tension).

Une information complexe est codée sur plusieurs bits.

Le mot de n bits ou word

Un ensemble de n bits consécutifs définit un mot au sens de l'ordinateur et est représenté par un ensemble n cellules électroniques consécutives.

Le nombre n est une caractéristique principale de l'ordinateur. On note parle octet pour un ensemble de 8 bits, de mot pour un ensemble de 16 bits, et de mot double pour un ensemble de 32 bits, de mot long pour un ensemble de 60, 64 bits.

(voir ci-dessous).

b_{n-1}	b _{n-2}	b_{n-3}	.	.	.	b ₃	b ₂	b₁	b ₀	Numéro du bit
0	1	1	1	0	0	1	1	0	1	Valeur du bit

↓
↓

Bit de poids fort
 Bit de poids faible

Figure 1-3 : Représentation binaire d'un mot.

- Le bit à l'extrême gauche (b_{n-1}) est le bit de poids fort (MSB-Most Significant Bit).
- Le bit à l'extrême droite (b_0) le bit de poids faible (LSB-Least Significant Bit).
- Dans le cas où $n = 2p$, de demi-mot de poids fort (ou le plus significatif) pour les p bits de gauche et de demi-mot de poids faible (ou le moins significatif) pour les p bits de droite.
- Par convention, les n bits d'un mot sont numérotés de 0 à n-1, de la droite vers la gauche
- Avec n bits, il est possible de représenter 2^n valeurs possibles.
- La valeur d'un bit dans un nombre binaire, appelé poids du bit, dépend de la position du bit en partant de la droite. Cette valeur croît d'une puissance de 2 en partant de la droite.

1.2.3 Les unités d'information

. La mesure de l'information binaire s'exprime en bits ou en octets (8 bits) avec les multiples courants suivants :

- Un quartet est un ensemble de 4 bits
- Un octet (byte) est un ensemble de 8 bits
- Un mot (word), un ensemble de 16 bits
- Un mot double, un ensemble de 32 bits
- Un mot long, un ensemble de plus de 32 bits

- 1 Kilo-octet (KO) = 10^3 octets
- 1 Méga-octet (MO) = 10^6 octets
- 1 Giga-octet (GO) = 10^9 octets
- 1 Tera-octet (TO) = 10^{12} octets
- 1 Pico-octet (PO) = 10^{15} octets
- 1 Exa-octet (EO) = 10^{18} octets
- 1 Zeta-octet (ZO) = 10^{21} octets
- etc.

N.B.: Le kilo informatique vaut 1024 et est ramené à 1000 depuis 1998 dans les systèmes informatiques.

1.3 LE CODAGE (REPRÉSENTATION) DE L'INFORMATION

Le système binaire fut adopté comme modèle de représentation (codage) des informations dans l'ordinateur. Le codage de l'information est donc la façon selon laquelle les différents types d'information naturelle (instructions, nombres, textes, images, vidéo, ...) sont décrits en binaire. En plus, le codage de l'information fait appel à la notion de cellule ou bascule (bistable), élément électronique ne pouvant prendre que deux niveaux de tensions différentes.

1.4 LE CODAGE DES DONNÉES NUMÉRIQUES

1.4.1 Introduction

Le codage des nombres a toujours existé dans les cultures. Les premières représentations utilisaient des traits (I, II, III, ...). Ces représentations ont engendré les symboles romains (I, II, III, IV, V, ...), puis les symboles arabes (1, 2, 3, 4, ...). Les différents symboles sont appelés des chiffres. Signalons que les représentations citées ne permettent pas de représenter le zéro (0) ou de faire des opérations arithmétiques.

D'où la nécessité de définir un système qui permet :

- d'écrire des expressions et effectuer des opérations ;
- de respecter une convention claire et précise ;
- de représenter le nombre 0, les fractions, etc.

Un tel système est appelé système de numération dans une base donnée. Ainsi, le système décimal (base 10) est utilisé par l'Homme tandis que l'ensemble des technologies numériques utilisent le système binaire (base 2) et ses systèmes associés : le système octal ou la base 8, le système hexadécimal (base 16), et le système DCB (Décimal Codé Binaire).

Signalons également la base quinaire (base 5, les Mayas), la base duodécimale (base 12, les Anglo-saxons et le système d'heure actuel), la base vicésimale (base 20, les Mayas), la base sexagésimale (base 60, les Sumériens et le système horaire actuel).

1.4.2 Ecriture (décomposition) d'un nombre dans une base

Considérons un système de numération à base b , un entier supérieur à 1, tout nombre N peut s'écrire sous la forme :

$$N = a_{n-1}b^{n-1} + a_{n-2}b^{n-2} + \dots + a_i b^i + \dots + a_0 b^0 + a_{-1}b^{-1} + \dots + a_{-i}b^{-i} + \dots + a_{-m}b^{-m}$$

avec $0 \leq a_i < b-1$.

On distingue donc la partie entière du nombre $PE = a_{n-1}b^{n-1} + a_{n-2}b^{n-2} + a_{n-3}b^{n-3} + \dots + a_0 b^0$, et sa partie fractionnaire $PF = x = a_{-1}b^{-1} + a_{-2}b^{-2} + a_{-3}b^{-3} + \dots$

Notes :

- Les valeurs a_i sont les restes issus des divisions successives de la partie entière du nombre par la base ;
- Les valeurs a_{-i} sont les parties entières obtenues à l'aide des multiplications successives de la partie fractionnaire dans la base ;
- Les b^i ou b^{-i} sont les poids respectifs des valeurs a_i ou a_{-i} en fonction des positions i ou $-i$ respectives ;

Le codage d'un nombre revient donc à l'écrire dans la base 2 (système binaire) sur un nombre de bits prédéfinis.

1.4.3 Codage des entiers naturels

Les nombres entiers positifs (entiers naturels, entiers non signés) sont représentés en binaire normale sur un nombre de bits prédéfini.

La suite de n bits : $a_{n-1} a_{n-2} \dots a_i \dots a_2 a_1 a_0$ avec $0 \leq a_i \leq 1$ représente, dans l'ordinateur, l'entier naturel N :

$$N = \sum_{i=0}^{n-1} a_i 2^i = a_{n-1}2^{n-1} + a_{n-2}2^{n-2} + \dots + a_i 2^i + \dots + a_2 2^2 + a_1 2^1 + a_0 2^0$$

La méthode de codage d'un entier naturel sur n bits consiste à effectuer des divisions successives par 2 jusqu'à un quotient nul et de garder les restes successifs pris du dernier au premier.

Le plus grand nombre que l'on peut représenter avec n bits est donc 1111.....111 (n bits de 1) soit,

$$2^{n-1} + 2^{n-2} + \dots + 2^i + \dots + 4 + 2 + 1 = 2^n - 1, \text{ le plus petit entier positif étant } 0.$$

On en déduit l'intervalle de représentation des entiers non signés qui est $[0, 2^n - 1]$.

Exemple 1: Coder le nombre 91_{10} en binaire sur 7 bits

Exemple 2 : Décoder le nombre binaire $(111110111101)_2$

1.4.4 Codage des entiers relatifs

1.4.4.1 Introduction

Dans le système décimal, le codage des entiers relatifs se fait sous la forme de signe et valeur absolue. En effet, le nombre positif est précédé du signe $+$ ($+7$) et le nombre négatif précédé

du signe – (-7). Par analogie, on peut adopter le même principe de représentation dans n'importe quelle base (binaire, octal, hexa, etc.) à la seule condition de préciser le mode de gestion du signe.

Les entiers relatifs sont les entiers signés pour l'ordinateur à savoir les entiers signés + ou entiers positifs et les entiers signés – ou entiers négatifs. Le problème ici est la représentation des signes + et – sous forme binaire dans l'ordinateur.

On convient donc de codifier le signe + par le bit 0 et le signe – par le bit 1, et réserver un bit, le bit de poids fort, au signe.

On utilise couramment trois méthodes pour représenter les nombres relatifs, à savoir :

-  Signe et valeur absolue ;
-  Complément à 1 (logique ou restreint) ;
-  Complément à 2 (arithmétique ou vrai)

1.4.4.2 Méthode du Signe et Valeur Absolue (SVA)

Le codage d'un entier relatif sur n bits consiste à garder après le bit de signe la traduction en binaire de la valeur absolue du nombre **sur (n-1) bits**. Si le nombre est négatif, le bit de signe vaut 1, 0 dans le cas contraire.

On montre que l'intervalle de représentation des nombres entiers signés est donc $-2^{n-1} + 1 \dots 2^{n-1} - 1$.

Par exemple, pour n égal à 6, les nombres relatifs que l'on peut représenter sont compris entre -31 et +31.

Notons cependant les particularités ci-après de cette méthode :

-  L'existence de deux représentations distinctes du zéro (-0 $\longrightarrow$ 10000...0 et +0 $\longrightarrow$ 00000...0) ;
-  La complexité des opérations d'addition et de multiplication du fait de la gestion du signe.
-  La nécessité de disposer au niveau matérielle de deux unités arithmétiques, une pour l'addition et l'autre pour la soustraction afin de faciliter la gestion du signe.

Exemple 5 : Représenter en signe et valeur absolue les nombres $+80_{10}$ et -45_{10} sur 8 bits.

1.4.4.3 Méthode du Complément à 1 (C1(N))

Le complément à 1 (complément logique, complément restreint) d'un nombre binaire est obtenu en changeant tous les 1 en 0 et tous les 0 en 1. De plus, le complément à 1 du complément à 1 d'un nombre binaire est le nombre binaire lui-même.

Pour coder un entier relatif sur n bits par la méthode du complément à 1, il faut :

-  Réserver le bit de poids fort pour le signe en respectant la convention de signe (0 pour + et 1 pour-) ;
-  Garder la traduction en binaire de la valeur absolue du nombre sur (n-1) bits, précédée du bit de signe 0, si ce dernier est un nombre de positif

- Prendre le complément à 1 de la traduction binaire de la valeur absolue sur (n-1) bits, précédée du bit de signe 1, si le nombre est négatif.

Ici encore, le problème des deux zéros (zéro négatif et du zéro positif) subsiste par contre la gestion opérations arithmétiques est simplifiée.

L'intervalle de représentation des entiers signés en complément à 1 est $-2^{n-1} + 1 \dots + 2^{n-1} - 1$.

1.4.4.4 Méthode du Complément à 2 (C2(N))

Le complément à 2 (complément arithmétique, complément vrai) d'un nombre binaire est obtenu en ajoutant 1 au complément à 1. De plus, le complément à 2 du complément à 2 d'un nombre binaire est le nombre binaire lui-même.

Pour coder un nombre relatif sur n bits par la méthode du complément à 1, le principe est le suivant :

- Réserver le bit de poids fort au signe en respectant la convention de signe (0 pour + et 1 pour-) ;
- Garder la traduction en binaire de la valeur absolue du nombre sur n bits si le nombre de positif ;
- Prendre le complément à 2 de la traduction binaire de la valeur absolue sur n bits si le nombre est négatif.

Le problème des deux zéros est résolu et l'on dispose d'un seul zéro non signé. En outre on note un dépassement de capacité dans le cas d'un nombre trop grand (positif) ou très petit (négatif).

L'intervalle de représentation des nombres en complément à 2 est -2^{n-1} à $+2^{n-1} - 1$.

On remarque que le codage du nombre $(-2^{n-1})_{10}$ est possible, par son opposé $(+2^{n-1})_{10}$ est impossible.

1.4.5 Codage des nombres réels

L'écriture d'un nombre réel R dans une base quelconque b est de la forme :

$$R = a_{n-1}b^{n-1} + a_{n-2}b^{n-2} + \dots a_i b^i + \dots + a_0 b^0 + a_{-1}b^{-1} + \dots a_{-i}b^{-i} + \dots + a_{-m}b^{-m}$$

avec $0 \leq a_i < b-1$.

Un nombre réel est donc constitué de deux parties :

- La partie entière du nombre(PE), $PE = a_{n-1}b^{n-1} + a_{n-2}b^{n-2} + a_{n-3}b^{n-3} + \dots + a_0b^0$
- La partie fractionnaire (PF), $PF = x = a_{-1}b^{-1} + a_{-2}b^{-2} + a_{-3}b^{-3} + \dots$

Le codage du réel dans une base donnée b est réalisée par la juxtaposition des représentations des parties entières et fractionnaires, séparées par la virgule. L'ordinateur ne possédant pas de virgule « numérique », il faut donc gérer la position de la virgule.

Deux méthodes sont possibles :

- la représentation (codage) en virgule fixe
- la représentation (codage) en virgule flottante

1.4.6 Codage de réels en virgule fixe

Le codage des réels en virgule fixe positionne de façon arbitraire la virgule, « entre deux bits ». Le principe est le suivant : garder de part et d'autre de la virgule les traductions binaires des parties entière et fractionnaire. Ainsi, la partie entière est représentée sur n bits et la partie fractionnaire sur m bits. Un bit de signe est également à prévoir.

Exemple: Coder le nombre réel $14,750_{10}$ en virgule fixe sur un ordinateur à 16 bits. On suppose que la « virgule » se situe entre les bits 3 et 4 de l'octet de poids faible. Pour la convention de signe, on prendra 0 pour + et 1 pour -.

La solution est : $14_{10} = 1110_2$ et $0,750_{10} = 0,11_2$, par suite le codage est :

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	1	1	1	0	1	1	0	0

Emplacement de la virgule

Figure 1.4: Représentation d'un réel en virgule fixe

Exemple : Convertir $10,675_{10}$ en binaire.

Le résultat est : $10,675_{10} = (1010,1011)_2$

1.4.7 Codage des réels en virgule flottante

1.4.7.1 Notion de virgule flottante

Le codage de réels en virgule flottante a pour objet de varier la position de la virgule du nombre d'une position à une autre. Tout nombre en virgule flottante est composé de deux parties principales:

- La valeur absolue du nombre, et
- La position de la virgule.

Cette représentation en virgule flottante s'apparente à la notation scientifique dans le système décimal où les nombres réels sont représentés par des nombres multipliés par des puissances de 10.

Et en notation mathématique, la forme générale d'un réel est : $R = s M B^E$

- s , le signe,
- M , la fraction (mantisse)
- B , la base,
- E , l'exposant.

Exemple :

$$60225 = 6,0225 \times 10^4$$

$$-(110,101)_2 = -(0,110101)_2 * 2^3$$

$$(0,00101)_2 = (0,101)_2 * 2^{-2}$$

$$0,00105 = 1,05 \times 10^{-3}$$

En définitive, le codage de réels en virgule flottante revient à représenter deux quantités : l'exposant et la fraction (mantisse). On évite de représenter la base B (base 2 pour tous les nombres).

La forme générale est :

Signe (s) de la mantisse	Exposant (E)	Mantisse normalisée(M)
--------------------------	--------------	------------------------

Figure 1.4 : Format d'un réel à virgule flottante

- L'exposant est représenté en complément à 2 ou sous la forme d'un entier biaisé, avec pour le biais 2^{n-1}
- La mantisse normalisée est en signe et valeur absolue.

1.4.7.2 Notion de mantisse normalisée

On dit que la mantisse est normalisée si le premier chiffre après la virgule est différent de 0 et le premier chiffre avant la virgule est égale à 0.

Par exemple, dans le système hexadécimal, le nombre $0,1A8 \times 16^4$ est une mantisse normalisée.

Dans le système binaire, on note deux conventions de normalisation.

-  Amener la virgule juste avant le 1 à l'extrême gauche i.e. Norme VAX (Virtual Address Indexing) $(0,101\dots)_2$
-  Amener la virgule juste après le 1 à l'extrême gauche, i.e. Norme IBM ou IEEE-754 $(1,101\dots)_2$.

1.4.7.3 Notation biaisée ou décalée

De ce qui précède, en complément à 2, l'intervalle des valeurs qu'on peut représenter sur n bits est : $-2^{(n-1)} \leq N \leq 2^{(n-1)} - 1$

Si on rajoute la valeur $2^{(n-1)}$ à chaque membre de cette inégalité, on obtient :

$$-2^{(n-1)} + 2^{(n-1)} \leq N + 2^{(n-1)} \leq 2^{(n-1)} - 1 + 2^{(n-1)}$$

$$0 \leq N + 2^{(n-1)} \leq 2^n - 1$$

On pose $N' = N + 2^{(n-1)}$ donc : $0 \leq N' \leq 2^n - 1$, dans ce cas on obtient des valeurs positives. La valeur 2^{n-1} s'appelle le biais ou le décalage

Avec l'exposant biaisé on a transformé les exposants négatifs à des exposants positifs en rajoutant à l'exposant la valeur 2^{n-1} .

Exposant Biaisé = Exposant réel + Biais

Ou

Exposant Réel = Exposant Biaisé - Biais

On peut utiliser les formules ci-dessous pour obtenir le codage du nombre décalé ou signé (de -2^{n-1} à $+2^{n-1} - 1$) :

Nombre signé → nombre non signé – biais.

(Nombre)_{binaire_décalé} = (nombre non signé – biais)_{binaire}.

On note :

-  La notation biaisée ou excédent -2^{n-1} est identique à celle du complément à 2, sauf que le bit de signe est inversé (0 pour négatif, et 1 pour positif).
-  La notation biaisée facilite la comparaison entre deux nombres. En effet, dans la convention classique, les nombres signés et non signés utilisent des méthodes de comparaison différente. Par exemple, soient 0100 et 1011 deux nombres, s'ils sont non signés alors $1011 > 0100$; s'ils sont signés alors $0100 > 1011$.

Exemple : Représentation biaisée des nombres 56_{10} et -77_{10} sur 8 bits.

1.4.7.4 Mise en œuvre du codage des réels

La méthode à suivre est la suivante :

-  Convertir le nombre en virgule fixe
-  Calculer la mantisse normalisée dans la forme indiquée
-  Déduire les quantités de mantisse, de signe, et d'exposant à l'aide de la formule de normalisation
-  Calculer l'exposant décalé et le convertir en binaire (exposant - biais = E) ou représenter l'exposant en complément à 2.
-  Donner la représentation finale du nombre.

1.4.7.5 Exemple de norme : Norme IEEE-754 ou Norme IBM

Dans la norme IBM, il est possible de coder le réel en virgule flottante simple précision (VFSP, Réel court) en virgule flottante double précision (VFDP, Réel long) et en virgule flottante précision étendue

- Réel en Virgule Flottante Simple Précision (VFSP)

Le nombre réel est représenté sur 32 bits avec 23 bits pour la mantisse (bit 0 à bit 22), 8 bits pour l'exposant (bit 23 à bit 30), 1 bit pour le signe (bit 31, le bit de poids fort).

31	30							23	22																				0
S	Exposant e7-e0								Mantisse m22-m0																				

Figure 2-5 : Réel en virgule flottante simple précision (Norme IEEE-754)

Exemple 12 : Représenter le nombre décimal 2,5 en virgule flottante dans la norme IEEE-4.

- Convertir le nombre en binaire : $2,5_{10} = 10,1_2$
- Adapter le nombre binaire dans la norme IEEE-754 et en déduire la mantisse et l'exposant biaisé : $10,1_2 = 1,01 \times 2^1$. D'où, la mantisse vaut 010..... ; et l'exposant biaisé vaut 1.
- Calculer l'exposant réel : Exposant réel = exposant biaisé + biais + bit du signe ; par suite, exposant réel = $1 + 127 = 128$
- Convertir l'exposant réel en binaire : Soit 128 en binaire donne 10000000 (exposant négatif)
- Déduire le bit de signe : Le nombre étant positif, $S = 0$.
- Donner le codage final du nombre.

[illegible]

Figure 2.6 : Représentation d'un réel dans la norme IEEE-754

Le nombre réel est représenté sur 64 bits avec 52 bits pour la mantisse (bit 0 à bit 51), 11 bits pour l'exposant (bit 52 à bit 60), 1 bit pour le signe (bit 61, le bit de poids fort).

[illegible]

La forme générale est donc : $R = (-1)^{\text{Signe}} * 2^{(\text{exposant}-1023)} * 1.\text{mantisse}$

Le nombre réel est représenté sur 80 bits avec 64 bits pour la mantisse (bit 0 à bit 63), 15 bits pour l'exposant (bit 64 à bit 78), 1 bit pour le signe (bit 79, le bit de poids fort).

[illegible]

La forme générale est donc : $R = (-1)^{\text{Signe}} * 2^{(\text{exposant} - 16383)} * 1.\text{mantisse}$

1.5 CODAGE DE L'INFORMATION ALPHANUMERIQUE (CARACTERES)

Le codage des caractères nécessite l'utilisation d'un code alphanumérique. Un code alphanumérique est donc un code binaire destiné à représenter les 10 chiffres décimaux, les lettres de l'alphabet, et un certain nombre de caractères spéciaux.

On constate que le nombre total d'éléments à coder doit être au moins supérieur à 36 (10 + 26 pour les lettres majuscules), soit au minimum 6 bits ($2^6 = 64$). Différents codes ont alors été définies. Signalons les plus usités (voir tableau):

Code alphanumérique

Le code alphanumérique à 6-bit ou code interne : ce code utilise 6 bits pour représenter les caractères. Il comporte 64 caractères, à savoir les chiffres décimaux de 000000 à 001001, les lettres majuscules, et quelques caractères spéciaux (: , = , , % , [, < , . ,) , , ; , \$, * , > ,] , (, ^ , ≤ , ≥ , → , ↓ , ↑ , > , < , ⊥ , à partir de 010001.

Code ASCII

Le code ASCII (American Standard Code for Information Interchange) à 7 bit plus 1 bit de parité (voir Annexe 1). Il ne prend pas en compte les caractères accentués. Cependant une extension ISO Latin permet de coder les caractères accentués.

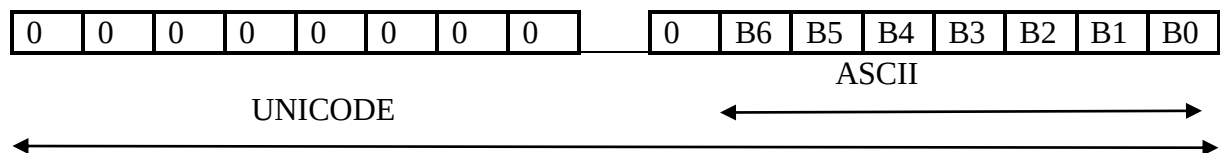
Code EBCDIC

Le code EBCDIC (Extended Binary Coded Decimal Internal Code) à 8-bit, utilisé par IBM.

Code UNICODE

Le code UNICODE est un code plus récent destiné à remplacer le code ASCII Il permet de manipuler tous les caractères possibles et dans toutes les langues (plus de 65000 caractères y compris les idéogrammes asiatiques).

C'est un code à 16/32 bits. Il nécessite l'utilisation de format type, le plus répandu est UTF_S. Sa représentation est telle que les caractères du répertoire ASCII sont représentés par les 7 bits de la représentation ASCII en poids faible, complétés par des 0 en poids forts :



1.6 AUTRES CODES BINAIRES

L'ordinateur ne traite pas que des nombres. Il est possible de traiter les lettres de l'alphabet, des températures, etc. Cette information doit être représentée sous forme binaire, d'où la nécessité de définir un code binaire

1.6.1 Code DCB

Le code DCB est un code binaire pondéré 8421 où tout chiffre décimal est représenté par les 4 bits correspondants. Le nombre décimal 970_{10} est codé en DCB par $100101110000_{\text{DCB}}$.

Remarquons que les codes en 8421 représentant les décimaux de 10 à 15 ne sont pas utilisés en DCB. Le code DCB est beaucoup utilisé dans les compteurs numériques, dans la transmission et la réception des données.

Notons aussi que le codage d'un nombre en DCB est différent de sa représentation en binaire. Des opérations arithmétiques sont possibles en DCB. Il faut cependant automatiquement un ajustement, dit « ajustement décimal » pour obtenir le bon résultat en cas de code erroné

1.6.2 Bases associées à la base 2

Les bases associées à la base 2 sont de la forme $B = 2^n$ où n est un entier naturel positif supérieur ou égal à 3. Elles permettent d'effectuer des opérations directes sur les bits et sur les symboles issus de ces bases. Ainsi le passage de la base associée à la base 2 se ramène à un éclatement des symboles de la base B en n bits par n bits du nombre binaire; et le passage de la base 2 à la base B se réalise par un regroupement de n bits par n -bits du nombre binaire.

Citons par exemple, les bases 8(système octal) et 16 (système hexadécimal).

1.6.3 Code réflecteur ou code GRAY

Rappelons que les systèmes numériques traitent des informations binaires. Toute information continue ou analogique doit être convertie en une information binaire avant d'être traitée par un système numérique, ordinateur par exemple. La conversion se fait au moyen d'un convertisseur analogique - numérique. Le code le plus adapté dans un tel cas est le code reflété (reflecté). La règle utilisée dans la définition d'un tel code est dans le codage d'un nombre donné, seul un bit change d'un nombre à un autre. Le tableau suivant donne les codes des nombres de 0 à 15.

Nombre Décimal	Code Réflecteur
0	0000
1	0001
2	0011
3	0010
4	0110
5	0111
6	0101
7	0100

8	1100
9	1101
10	1111
11	1110
12	1010
13	1011
14	1001
15	1000

Nombre décimal	0	1	2	3	4	5	6	7
Code réflecté	0000	0001	0011	0010	0110	0111	0101	0100

Nombre décimal	8	9	10	11	12	13	14	15
Code réflecté	1100	1101	1111	1110	1010	1011	1001	1000

1.7 CALCUL BINAIRE

1.7.1 Addition binaire des entiers non signés

L'addition binaire est sans doute la première opération arithmétique à étudier. La table d'addition binaire est la suivante :

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 0 \text{ plus 1 de retenue dans la position suivante.}$$

De plus, l'addition binaire ne diffère pas fondamentalement de l'addition décimale. Tout se ramène à une gestion de la retenue comme c'est le cas en décimal.

Exemple : Additionner les nombres binaires 00000100 et 00000101

La progression de la retenue se fait d'une colonne à une autre. De plus, les circuits électroniques nécessaires à la mise en œuvre de l'addition sont très simples et faciles. La difficulté réside dans

la manipulation des 0 et des 1. C'est pourquoi d'autres formes de représentation sont utilisés (systèmes associés au binaire), l'ordinateur effectuant toujours son opération en binaire.

N.B. : Notion de dépassement de capacité (Overflow)

L'addition de deux nombres à n -bits peut nécessiter $n + 1$ bits pour représenter la somme. Cette représentation serait alors trop large pour tenir dans le mot mémoire de l'ordinateur. C'est le concept de dépassement de capacité ou overflow.

Exemple : Un ordinateur travaille avec des mots de 6-bit. A l'aide de cet ordinateur, additionner les nombres 26_8 et 45_8 d'une part et 52_8 et 61_8 d'autre part.

Dans le deuxième cas on aboutit à une retenue dans la 7^{ème} position. Ce qui est supérieur au mot de 6-bit. Il y a donc dépassement de capacité et l'ordinateur ne pourra pas représenter cette somme.

1.7.2 Soustraction binaire des entiers signés

La soustraction de deux nombres binaires peut se ramener à l'addition d'un des nombres avec l'opposé de l'autre (opposé ou complément). Ainsi, avec un même circuit logique, on peut effectuer l'addition et la soustraction binaires. Elle se fait sur la base des modes de représentation des entiers négatifs.

Signe et valeur absolue

Soit à effectuer la soustraction des nombres $+7$ et -6 . On a

$+7$	0111
-6	1110
----	-----
$+1$	?101

Le résultat est difficile à interpréter du fait de la nécessité de définir un mode de gestion particulier du bit de signe.

Complément à b (2).

La soustraction de deux nombres signés M et N , $(M - N)$, M est positif, en complément à 2 se fait comme suit :

- Trouver le complément à b (2) de N et l'ajouter à M ;
- Vérifier si le résultat admet une retenue de dépassement :
 - Si oui, ignorer cette retenue ;

Exemple : Effectuer en complément à 2 , les opérations suivantes :

- ♦ $(72532 - 3250)_{10}$
- ♦ $(3250 - 72532)_{10}$.

N.B.: Le dépassement de capacité peut subvenir dans le cas d'un nombre trop grand (positif) ou très petit (négatif).

Complément à (b-1) (1)

La soustraction de deux nombres signés M et N ($M - N$), M est positif, en complément à 1, se fait comme suit :

- c) Trouver le complément à 1 de N et l'ajouter à M ;
- d) Vérifier si le résultat admet une retenue de dépassement :
 - Si oui, l'ajouter au chiffre de poids faible;

Ainsi les opérations d'addition et de soustraction en complément à 1 nécessitent la prise en compte de la retenue appelée « **retenue circulaire** ». Il s'agit de la retenue provenant du bit de poids fort (bit du signe) qui doit être ajouté au bit de poids faible.

Justifions cette notion à l'aide de l'exemple ci-dessous.

Exemple : Soit à effectuer sur un ordinateur à 8-bits, les opérations suivantes portant sur les nombres -33_{10} et $+21_{10}$, puis -33_{10} et -21_{10}

a)

```

-33  11011110
+21  00010101
-----
-12  11110011 = -1210
retenue
  
```

b)

```

-33  11011110
-21  11101010
-----
-54  1)11001000 = -5510 avec une
      
```

Le résultat a) est correct, par contre celui de b) nécessite de rajouter la retenue 1 au résultat pour obtenir un résultat correct. Ainsi,

```

-33  11011110
-21  11101010
-----
-54  1)11001000 = -5510
      ↘
        +1 (retenue circulaire)
        -----
        11001001 = -5410
      
```

N.B.1 : Le dépassement de capacité peut subvenir dans le cas d'un nombre trop grand (positif) ou très petit (négatif).

N.B. 2 : L'addition de nombres de signe contraire n'engendre jamais de dépassement de capacité.

1.7.3 Multiplication binaire

La table de multiplication binaire est la suivante :

$$0 \times 0 = 0 \quad 0 \times 1 = 0$$

$$1 \times 0 = 0 \quad 1 \times 1 = 1$$

La multiplication de nombres entiers consiste en des décalages vers la gauche des bits du multiplicande et des additions. En général, la multiplication est réalisée à l'aide d'un programme.

Exemple : Multiplier le nombre 00001111_2 par 00001010_2 .

00001111

$\times 00001010$

00000000

000011110

0000000000

00001111000

$$00010010110 = 2^7 + 2^4 + 2^2 + 2^1 = 128 + 22 = 150_{10} \quad (15 \times 10 = 150)$$

1.7.4 Division binaire

La division binaire est semblable à la division décimale. La table de division est la suivante :

0 / 0 et 1 / 0 impossible ou indéterminé

Exemple : Diviser 27_{10} par 4_{10} .

On a : $27_{10} = 00011011_2$ et $4_{10} = 100_2$.

D'où :

110'1 | 100

101 110,11 $\longrightarrow$ $110,11_2 = 6,75 = 27/4$

110

100

0

N.B. : Signalons que dans les ordinateurs, la multiplication et la division se font à l'aide de programmes

1.7.5 Addition et soustraction en DCB

Exemple : Additionner en DCB les nombres 60_{10} et 55_{10} .

On a :

$60_{10} = 01100000_{DCB}$

$+55_{10} = 01010101_{DCB}$

10110101_{DCB}

Le résultat est incorrect. Le code DCB 1011 n'existe pas. Il faut donc réaliser une correction. La règle consiste à ajouter 0110 (6_{10}) au(x) code(s) incorrects.

$$\begin{array}{r} \text{D'où} \quad 1011\ 0101_{\text{DCB}} \\ +0110 \\ \hline 1\ 0001\ 0101_{\text{DCB}} \end{array} \longrightarrow 115_{10}$$

Pour la soustraction, il faut retrancher 6_{10} au résultat au niveau du code erroné.

1.7.6 Addition de réels en virgule flottante

Les opérations arithmétiques en virgule flottante sont très complexes. Dans le cas d'une addition, les étapes à suivre sont :

- Aligner les virgules des deux nombres par un décalage à droite du nombre avec le plus petit exposant jusqu'à ce que les exposants soient égaux ;
- Effectuer la somme ;
- Tronquer ou arrondir le résultat ;
- Normaliser le résultat.

Notion de troncation

Tronquer un nombre, c'est ignorer un certain nombre de bits que le mot mémoire ne peut pas contenir.

Par exemple, l'addition des deux nombres $0,111010100 \cdot 2^4$ et $0,111000111 \cdot 2^2$ sur 9 bits conduit à une troncation du résultat.

$$\begin{array}{r} 0,111010100 \cdot 2^4 \\ 0,111000111 \cdot 2^2 \\ \hline \end{array} \qquad \begin{array}{r} 0,111010100 \cdot 2^4 \\ 0,00111000111 \cdot 2^4 \\ \hline 1,00100010111 \cdot 2^4 \end{array}$$

Le résultat comporte plus de bits que le nombre de bits des nombres originaux. A cause de la taille fixe du mot mémoire, le résultat peut être tronqué à 9 bits après la normalisation. Par suite, le résultat final serait $0,100100010 \cdot 2^5$. La troncation produit toujours un résultat inférieur ou égal au résultat exact.

Notion d'arrondi

Le mode arrondi ajoute 1 au bit de poids faible lorsque le bit que l'on ne peut pas représenter vaut 1. C'est la notion d'arrondi.

N.B.: Dans l'exemple précédent, le résultat arrondi est $0,100100011 \cdot 2^5$